Legal Document Summarization using LoRA (PEFT)

📌 Project Overview

In this task, I explored domain-specific fine-tuning on legal documentation. Standard language models often struggle with complex legal vocabulary and long, statutory sentences.

Instead of training a massive model from scratch or updating all billions of weights (which is slow and memory-heavy), I used **Parameter-Efficient Fine-Tuning (PEFT)** with **Low-Rank Adaptation (LoRA)** on a quantized base model (QLoRA). This allowed me to train the model quickly on a single free Google Colab GPU (T4).

🛠️ Tech Stack & Tools

- **Base Model:** `Qwen/Qwen2.5-1.5B-Instruct`
- **Fine-Tuning Technique:** LoRA (via Hugging Face `peft`)
- **Quantization:** 4-bit NormalFloat (NF4) using `bitsandbytes`
- **Frameworks:** PyTorch, Transformers, Datasets, Accelerate
- **Compute:** Google Colab (Tesla T4 GPU)

📊 Dataset & Preparation

- **Dataset Used:** `FiscalNote/billsum` (contains U.S. Congressional and state legislative bills paired with summaries).
- **Preprocessing:** Formatted the data using standard ChatML instruction tags (`<|im_start|>system`, `user`, and `assistant`) so the model clearly distinguishes between instructions, the raw legal bill, and the target plain-English summary.

⚙️ Hyperparameters & Training Setup

Parameter	Value	Reason / Note
Base Model Precision	4-bit (QLoRA)	To fit the base model in ~1.2 GB of GPU RAM
LoRA Rank (r)	16	Controls the rank dimension of trainable adapter matrices
LoRA Alpha (α)	32	Scaling factor for the adapter updates
Target Modules	`q_proj`, `k_proj`, `v_proj`, `o_proj`	Added adapters to core attention projection layers
Trainable Parameters	0.2815%	Kept base weights frozen to avoid catastrophic forgetting
Learning Rate	`3e-4` (Cosine Decay)	Smooth convergence

Parameter	Value	Reason / Note
Batch Size & Accumulation	Batch size 4, Accumulation 2	Effective batch size of 8

📈 Training Progress

The model converged smoothly during training across steps:

- **Step 10:** Loss `1.6340`
- **Step 30:** Loss `1.4320`
- **Step 60:** Loss `1.4364`

🧪 Results & Model Inference

After fine-tuning, I reloaded the base model in evaluation mode, dynamically attached the trained PEFT adapter (`peft_legal_final`), and tested it on unseen legislative text from the test set.

Sample Output:

- **Raw Legal Bill Text (Input):** Dense statutory text regarding water resource infrastructure and environmental authorizations.
- **Fine-Tuned Model Summary (Output):**

  "Environmental Infrastructure: Amends the Water Resources Development Act of 1992 to provide $20 million for each of the following projects:

  - For the City of Jackson, Mississippi, to construct an alternative water supply system and a project for the elimination or control of combined sewer overflows
  - For the Town of Manchester, New Hampshire, to construct an alternative water supply system and a project for the elimination or control of combined sewer overflows;
  - For the City of Atlanta, Georgia, to construct an alternative water supply system and a project for the elimination or control of combined sewer overflows; and
  - For the cities of Paterson, Passaic County, and Newark, New Jersey, to construct an alternative water”

✨ Key Learnings & Highlights

1. **Efficiency:** Only trained 0.2815% of the parameters, saving hours of training time and keeping GPU memory usage well under 5 GB.

2. **Lightweight Deliverable:** The output adapter weight file (`adapter_model.safetensors`) is very less, making it easy to share and load without downloading the full model weights again.
3. **Domain Adaptation:** The fine-tuned model learnt to structure legal text into clear, bulleted summaries highlighting key financial numbers and obligations.